

Relatório sobre a atividade “16 - Prática: Docker e Containers para Aplicações (III)”

Lucas Gabriel Arenhardt

1. Introdução

Neste módulo foi apresentado o Docker, uma plataforma para criar, executar e gerenciar aplicativos em containers. É algo muito útil, pois torna a execução do aplicativo independente da máquina utilizada. Os conceitos aprendidos estão apresentados a seguir.

2. Conceitos

2.1. Container

É uma unidade isolada utilizada para empacotar aplicações com todos os arquivos necessários para rodá-las corretamente. Assim, não há problema de incompatibilidade quando a aplicação for utilizada em outro ambiente.

2.2. Docker

O Docker é uma plataforma que facilita a criação, execução e gerenciamento de containers. Ele fornece diversas ferramentas para empacotar aplicações e suas dependências.

A estrutura padrão dos comandos docker é: **docker {objeto} {ação} {opções}**. Os principais comandos são:

- **docker [objeto] ls:** lista todos os objetos do tipo informado ativos. Caso utilizada a configuração -a, também mostra os inativos;
- **docker [objeto] rm:** exclui um objeto do tipo informado;
- **docker [objeto] inspect:** mostra várias informações úteis sobre o objeto informado;
- **docker container create:** cria um novo container. Algumas opções são: nomear o container com -name, -i para torná-lo interativo, -v para utilizar um volume, dentre outras. Deve-se especificar a imagem na qual o container será criado;
- **docker container rename:** renomeia um container;
- **docker container start/stop:** inicia ou encerra um container;
- **docker container run:** cria e conecta a um container;

- **docker container run -v [host_path]:[container_path]:** monta um volume do sistema de arquivos do Host no container Docker. Um volume é uma área de armazenamento persistente e gerenciável que pode ser usada para armazenar dados que precisam ser preservados e compartilhados entre containers;
- **docker run -p [hostPort]:[containerPort]:** cria e inicia um container mapeando uma porta na máquina Host para uma porta no container;
- **docker container attach:** conecta a um container;
- **docker container cp:** copia um container;
- **docker container commit:** cria uma nova imagem a partir do estado atual de um containers;
- **docker logs [container]:** exibe os logs de um container Docker;
- **docker image save:** salva uma imagem Docker no formato ‘.tar’;
- **docker image load:** Importa uma imagem Docker de um arquivo ‘.tar’;
- **docker image pull:** baixa uma imagem Docker do Docker Hub;
- **docker image tag:** adiciona uma tag a uma imagem Docker existente;
- **docker image push:** envia uma imagem Docker para um repositório remoto;
- **docker image build:** constrói uma imagem Docker a partir de uma DockerFile.

2.3. Exemplo utilizando o Docker

Neste exemplo, cria-se um container chamado “teste” a partir da imagem “alpine”. Dentro do container, é criado um arquivo de texto. As alterações feitas são salvas em uma nova imagem Docker, que é exportada para um arquivo “.tar”. Em seguida, a imagem original é removida e a imagem salva é recarregada a partir do arquivo .tar. Finalmente, um novo contêiner chamado “testeDoTar” é iniciado usando a imagem carregada.

```
$ docker container run --name teste -it alpine sh
/ # mkdir teste
/ # cd teste
/teste # touch lucas.txt
/teste # echo "TESTE DE ESCRITA" > lucas.txt
/teste # cat lucas.txt
TESTE DE ESCRITA
```

```
$ docker container commit teste teste-img
```

```
$ docker image save -o teste.tar teste-img
```

```
$ docker image rm teste-img
```

```
lucasarenhardt@pop-os:~$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	edbf74c41f8	2 weeks ago	78.1MB
alpine	latest	324bc02ae123	4 weeks ago	7.8MB

```
$ docker image load -i teste.tar
```

```
lucasarenhardt@pop-os:~$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
teste-img	latest	3250b59c29c9	6 minutes ago	7.8MB
ubuntu	latest	edbf74c41f8	2 weeks ago	78.1MB
alpine	latest	324bc02ae123	4 weeks ago	7.8MB

```
$ docker container run -it --name testeDoTar teste-img sh
```

```
lucasarenhardt@pop-os:~$ docker container run -it --name testeDoTar teste-img sh
```

```
/ # ls
bin    etc    lib    mnt    proc   run    srv    teste  usr
dev    home  media  opt    root   sbin   sys    tmp    var
/ # cd teste
/teste # ls
lucas.txt
/teste # cat lucas.txt
TESTE DE ESCRITA
/teste #
```

3. Conclusão

Este módulo apresentou o conceito de containers e o uso do Docker para gerenciá-los. As aulas demonstraram como o Docker facilita a criação e o gerenciamento de ambientes isolados, assegurando que as aplicações funcionem de maneira consistente em qualquer sistema. O conhecimento adquirido é essencial para melhorar a eficiência e a portabilidade das aplicações em diferentes contextos de desenvolvimento.